

LAB 3 AVPR: Features descriptors, local feature matching

Author: Oscar Cubeles

```
import os
import cv2
import numpy as np
import seaborn as sns

from matplotlib import pyplot as plt

from skimage.feature import hog
from skimage import feature, io, exposure

from sklearn.preprocessing import LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
```

TASK 1: Feature Extraction with SIFT

```
img1_path = 'raj_dancing.jpg'
img2_path = 'sheldon_dancing.jpg'
image1 = cv2.imread(img1_path, cv2.IMREAD_GRAYSCALE)
image2 = cv2.imread(img2_path, cv2.IMREAD_GRAYSCALE)

sift = cv2.SIFT_create()

keypoints1, descriptors1 = sift.detectAndCompute(image1, None)
keypoints2, descriptors2 = sift.detectAndCompute(image2, None)

# Display detected keypoints on the original images
print(f"Number of keypoints detected in {img1_path}: {len(keypoints1)}")
print(f"Number of keypoints detected in {img2_path}: {len(keypoints2)}")

# Draw keypoints on the images
img1_with_keypoints = cv2.drawKeypoints(image1, keypoints1, None,
color=(0, 255, 0), flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
img2_with_keypoints = cv2.drawKeypoints(image2, keypoints2, None,
color=(0, 255, 0), flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

# Visualize the images with keypoints
```

```
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

axes[0].imshow(img1_with_keypoints)
axes[0].set_title(f'{img1_path} - SIFT Keypoints Detected')
axes[0].axis('off')

axes[1].imshow(img2_with_keypoints)
axes[1].set_title(f'{img2_path} - SIFT Keypoints Detected')
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

Number of keypoints detected in raj_dancing.jpg: 1082
Number of keypoints detected in sheldon_dancing.jpg: 551

raj_dancing.jpg - SIFT Keypoints Detected



sheldon_dancing.jpg - SIFT Keypoints Detected



At first glance the results can be overwhelming, in both images the number of detected keypoints is considerably high, in both cases more than 500 detections. When plotted on the image, the results are difficult to analyse. Therefore, in order to obtain a more visual result, I will proceed to filter the most relevant keypoints by response strength and plot them.

```
def get_most_relevant_keypoints(image, percentage=20):

    # Detect SIFT keypoints and descriptors
    sift = cv2.SIFT_create()
    keypoints, descriptors = sift.detectAndCompute(image, None)

    if len(keypoints) == 0:
```

```

        return [], np.array([]), np.array([])

    # Get response values
    responses = np.array([kp.response for kp in keypoints])

    # Calculate how many keypoints to keep
    num_top = max(1, int(len(keypoints) * percentage / 100))

    # Get indices of top keypoints
    top_indices = np.argsort(responses)[-num_top:]

    # Extract top keypoints, descriptors, and responses
    top_keypoints = [keypoints[i] for i in top_indices]
    top_descriptors = descriptors[top_indices] if descriptors is not
None else np.array([])
    top_responses = responses[top_indices]

    print(f"Selected {len(top_keypoints)} most relevant keypoints out
of {len(keypoints)} total")

    return top_keypoints, top_descriptors, top_responses

top_kp1, top_desc1, top_resp1 = get_most_relevant_keypoints(image1)
top_kp2, top_desc2, top_resp2 = get_most_relevant_keypoints(image2)

# Visualize the results
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Draw only the most relevant keypoints
img1_top = cv2.drawKeypoints(image1, top_kp1, None, color=(255, 0, 0),

flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
img2_top = cv2.drawKeypoints(image2, top_kp2, None, color=(255, 0, 0),

flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

axes[0].imshow(img1_top)
axes[0].set_title(f'Top 20% Most Relevant Features - Image 1
({len(top_kp1)} keypoints)')
axes[0].axis('off')

axes[1].imshow(img2_top)
axes[1].set_title(f'Top 20% Most Relevant Features - Image 2
({len(top_kp2)} keypoints)')
axes[1].axis('off')

plt.tight_layout()
plt.show()

```

Selected 216 most relevant keypoints out of 1082 total
Selected 110 most relevant keypoints out of 551 total

Top 20% Most Relevant Features - Image 1 (216 keypoints)



Top 20% Most Relevant Features - Image 2 (110 keypoints)



In my opinion, discarding less relevant features makes the results easier to interpret and allows to see what SIFT detects as more relevant features and keypoints in the image. For instance, on the image on the left (Raj), we can see that there are several keypoints around its shoulders and face.

However, this approach has several limitations such as ignoring spatial distribution of features, meaning that the current code may detect features from the same region that have high response values and returning keypoints that belong on close areas, dismissing information on other zones of the image.

We can see that for instance as the image of Raj on the left barely detects any keypoints on the central part of the image and the same happens on the image on the right, where the central and bottom parts barely have keypoints. To aim to solve this, I will divide the image in a grid and detect features on each grid cell.

```
def get_spatially_distributed_features(image, keypoints_per_cell=2,
grid_size=5):
    sift = cv2.SIFT_create()
    keypoints, descriptors = sift.detectAndCompute(image, None)

    if len(keypoints) == 0:
        return [], np.array([])

    h, w = image.shape
    grid_h, grid_w = h // grid_size, w // grid_size

    # Divide image into grid and select multiple keypoints from each
```

```

cell
    selected_kp = []
    selected_desc = []

    for i in range(grid_size):
        for j in range(grid_size):
            # Define grid boundaries
            y1, y2 = i * grid_h, (i + 1) * grid_h
            x1, x2 = j * grid_w, (j + 1) * grid_w

            # Find keypoints in this grid cell
            cell_kp = []
            cell_indices = []
            for idx, kp in enumerate(keypoints):
                if x1 <= kp.pt[0] < x2 and y1 <= kp.pt[1] < y2:
                    cell_kp.append(kp)
                    cell_indices.append(idx)

            # Select top N keypoints from this cell
            if cell_kp:
                responses = [kp.response for kp in cell_kp]
                # Get indices of top keypoints (sorted by response)
                top_indices = np.argsort(responses)[-
keypoints_per_cell:]

                # Add the top keypoints from this cell
                for idx in top_indices:
                    selected_kp.append(cell_kp[idx])

            selected_desc.append(descriptors[cell_indices[idx]])

    return selected_kp, np.array(selected_desc)

def visualize_keypoints_with_grid(images, keypoints_list, titles,
grid_size=5, color=(0, 255, 255)):
    fig, axes = plt.subplots(1, len(images), figsize=(15, 6))

    # Handle single image case
    if len(images) == 1:
        axes = [axes]

    for idx, (image, keypoints, title) in enumerate(zip(images,
keypoints_list, titles)):
        # Draw keypoints
        img_with_keypoints = cv2.drawKeypoints(image, keypoints, None,
color=color,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

        axes[idx].imshow(img_with_keypoints)

```



```

axes[idx].set_title(f'{title} ({len(keypoints)} keypoints)')
axes[idx].axis('off')

# Draw grid lines
h, w = image.shape
for i in range(1, grid_size):
    axes[idx].axhline(y=i * h // grid_size, color='red',
linestyle='--', alpha=0.5, linewidth=1)
    axes[idx].axvline(x=i * w // grid_size, color='red',
linestyle='--', alpha=0.5, linewidth=1)

plt.tight_layout()
plt.show()

# Print results
for title, keypoints in zip(titles, keypoints_list):
    print(f"{title}: {len(keypoints)} keypoints selected")

selected_kp1, selected_desc1 =
get_spatially_distributed_features(image1, keypoints_per_cell=5)
selected_kp2, selected_desc2 =
get_spatially_distributed_features(image2, keypoints_per_cell=5)

images = [image1, image2]
keypoints_list = [selected_kp1, selected_kp2]
titles = ['Spatially Distributed Features - Image 1', 'Spatially
Distributed Features - Image 2']

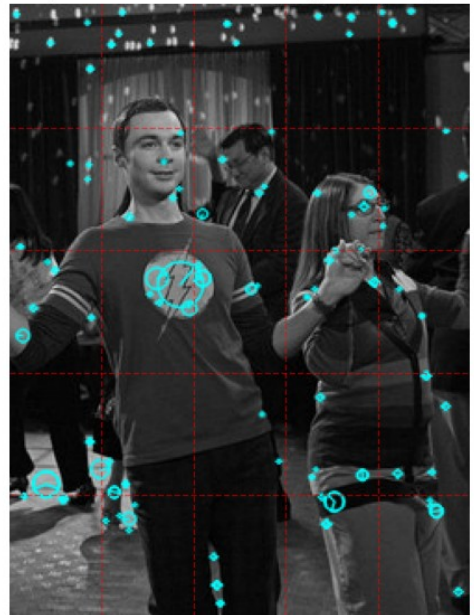
visualize_keypoints_with_grid(images, keypoints_list, titles)

```

Spatially Distributed Features - Image 1 (125 keypoints)



Spatially Distributed Features - Image 2 (122 keypoints)



Spatially Distributed Features - Image 1: 125 keypoints selected
Spatially Distributed Features - Image 2: 122 keypoints selected

With this grid-like approach on both images we can see what is relevant on each grid cell. For instance, if we compare the current results with the previous ones on the image on the right, here we have several keypoints on sheldons t-shirt which is a distinctive part of the image while previously we just had one. Also, there are several keypoints on Amy's face and on her glasses.

I believe that this approach allows to better grasp the important parts of the image in a more high level view. However, for further analysis needs to be done to determine if this is rather a simplistic approach or not.

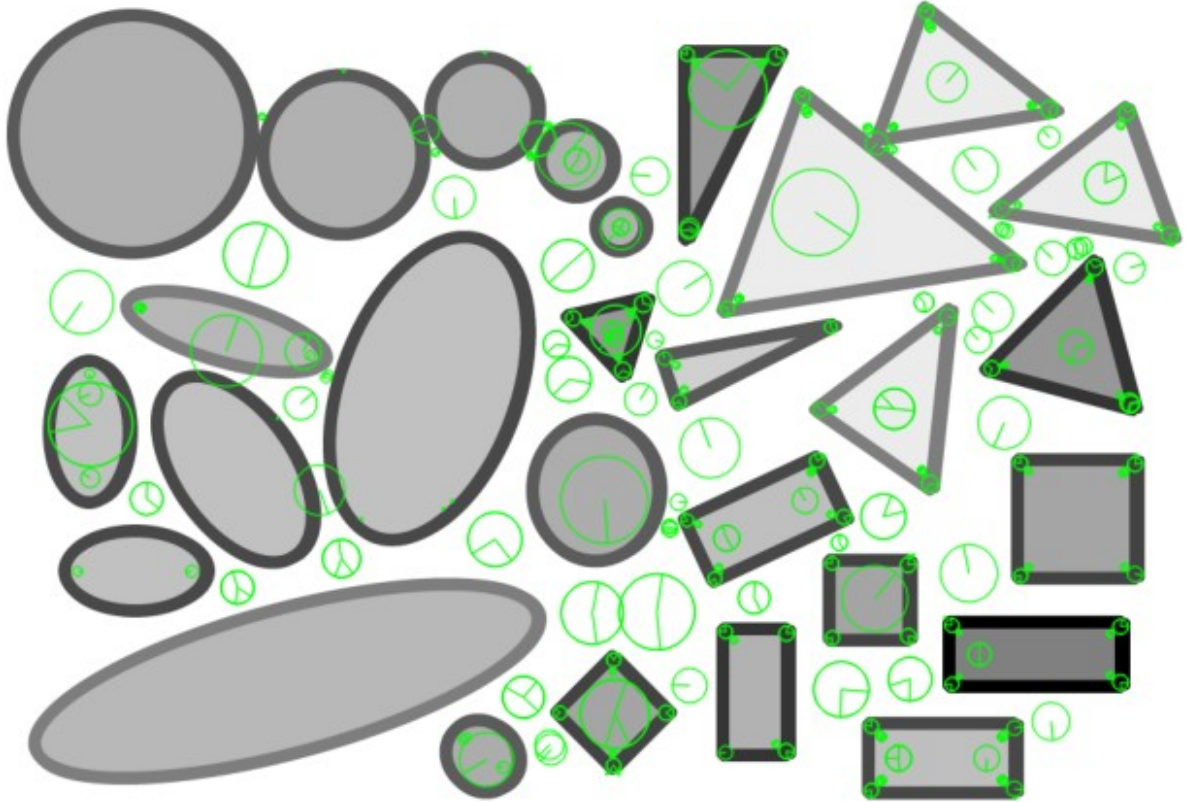
```
img3_path = 'geometries.png'
image3 = cv2.imread(img3_path, cv2.IMREAD_GRAYSCALE)
sift = cv2.SIFT_create()
keypoints3, descriptors3 = sift.detectAndCompute(image3, None)

# Display detected keypoints on the original image
print(f"Number of keypoints detected in {img3_path}: {len(keypoints3)}")
img3_with_keypoints = cv2.drawKeypoints(image3, keypoints3, None,
color=(0, 255, 0), flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

# Visualize the image with keypoints
plt.figure(figsize=(8, 6))
plt.imshow(img3_with_keypoints)
plt.title(f'{img3_path} - SIFT Keypoints Detected')
plt.axis('off')
plt.show()
```

Number of keypoints detected in geometries.png: 397

geometries.png - SIFT Keypoints Detected



I aimed to put a simpler image to see what SIFT detects. In most cases, SIFT detects the corners of the different shapes such as squares, rectangles and triangles. However SIFT seem to struggle with oval shapes. Also, it is curious how it detects keypoints in areas that do not have any shape and are blank.

Task 2: Local Feature Matching with SIFT

```
def match_sift_features(image1, image2, ratio_threshold=0.5,
                        visualize=True):

    # Create SIFT detector
    sift = cv2.SIFT_create()

    # Detect keypoints and compute descriptors for both images
    keypoints1, descriptors1 = sift.detectAndCompute(image1, None)
    keypoints2, descriptors2 = sift.detectAndCompute(image2, None)

    if descriptors1 is None or descriptors2 is None:
        print("No descriptors found in one or both images")
        return [], keypoints1, keypoints2, descriptors1, descriptors2

    # Create a Brute-Force Matcher
```



```

bf = cv2.BFMatcher()

# Match descriptors between the two images
matches = bf.knnMatch(descriptors1, descriptors2, k=2)

# Apply ratio test to obtain good matches
good_matches = []
for match_pair in matches:
    if len(match_pair) == 2:
        m, n = match_pair
        if m.distance < ratio_threshold * n.distance:
            good_matches.append(m)

print(f"Found {len(good_matches)} good matches")

# Visualize if requested
if visualize:
    # Draw matches on a new image
    matched_image = cv2.drawMatches(image1, keypoints1, image2,
keypoints2, good_matches, None,

flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

    # Display the matched image
    plt.figure(figsize=(15, 8))
    plt.imshow(matched_image)
    plt.title(f'SIFT Feature Matching - {len(good_matches)}
matches')
    plt.axis('off')
    plt.show()

    return good_matches, keypoints1, keypoints2, descriptors1,
descriptors2

sagrada_familia_path = 'sagrada_familia.jpg'
sagrada_familia_path2 = 'sagrada_familia2.jpg'
sagrada_familia_path3 = 'sagrada_familia3.png'
sagrada_familia_path4 = 'sagrada_familia4.png'

# Load images
image1 = cv2.imread(sagrada_familia_path, cv2.IMREAD_GRAYSCALE)
image2 = cv2.imread(sagrada_familia_path2, cv2.IMREAD_GRAYSCALE)
image3 = cv2.imread(sagrada_familia_path3, cv2.IMREAD_GRAYSCALE)
image4 = cv2.imread(sagrada_familia_path4, cv2.IMREAD_GRAYSCALE)

print(f"{sagrada_familia_path} shape: {image1.shape}")
print(f"{sagrada_familia_path2} shape: {image2.shape}")
print(f"{sagrada_familia_path3} shape: {image3.shape}")
print(f"{sagrada_familia_path4} shape: {image4.shape}")

```

```
# Match features
matches, kp1, kp2, desc1, desc2 = match_sift_features(image1, image2)

# Match features on similar size images
matches, kp1, kp2, desc1, desc2 = match_sift_features(image3, image4)

# Match features on similar size and higher ratio
matches, kp1, kp2, desc1, desc2 = match_sift_features(image3, image4,
ratio_threshold=0.7)

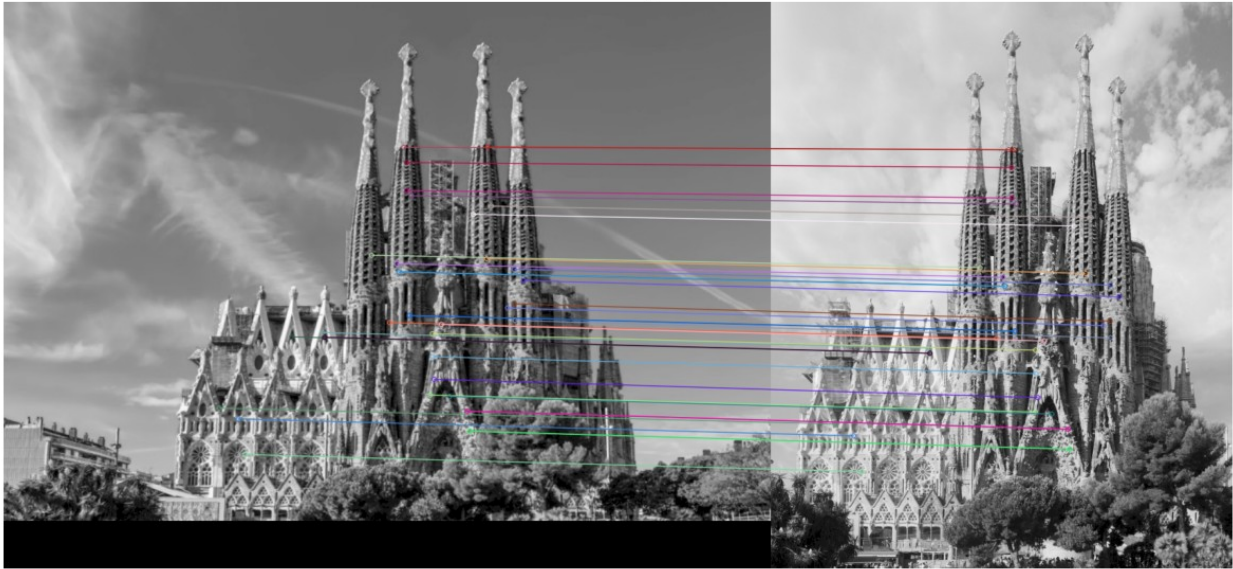
sagrada_familia.jpg shape: (4099, 2699)
sagrada_familia2.jpg shape: (673, 1009)
sagrada_familia3.png shape: (721, 1067)
sagrada_familia4.png shape: (787, 642)
Found 34 good matches
```

SIFT Feature Matching - 34 matches



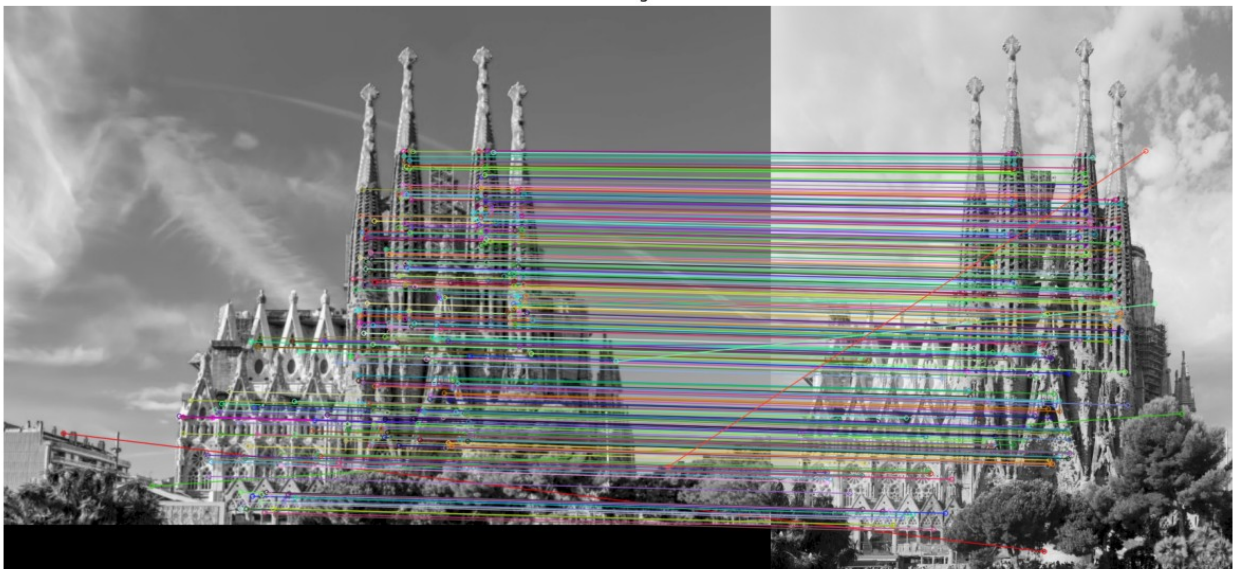
Found 36 good matches

SIFT Feature Matching - 36 matches



Found 310 good matches

SIFT Feature Matching - 310 matches



Looking at the results, we can observe that increasing or decreasing image sizes to make them more similar does not have a high affect on the number of matches. We can observe that from the previous examples where there was only an increase from 34 to 36 matches.

Additionally, I added a higher threshold to see the results. What can be observed is that by increasing the threshold ratio I obtaine more matches in the ratio test. This leads to having more matches but less reliable ones and it may be prone to obtain more false positive matches. For instance, we can observe one false match from the thirs plot that goes from the bottom part of the image on the left to a cloud on the image on the right.

However, I would say that with both ratio values (0.5 and 0.7) the results are quite satisfactory and overall matches come from similar areas on the images.

Task 3: Local Binary Pattern (LBP) Feature Extraction

```
def extended_lbp(image, radius, neighbors):
    lbp = feature.local_binary_pattern(image, neighbors, radius,
    method='uniform')
    return lbp

def apply_lbp_and_display(original_image):
    # Convert the image to a numpy array
    if not isinstance(original_image, np.ndarray):
        original_image = np.array(original_image)

    # Ensure image is in the range [0, 1]
    normalized_image = exposure.rescale_intensity(original_image,
    in_range=(0, 1))

    # Convert the image to uint8 for LBP computation
    image_uint8 = (normalized_image * 255).astype(np.uint8)

    # Circular LBP with (8, 1) neighborhood
    lbp_8_1 = extended_lbp(image_uint8, radius=1, neighbors=8)

    # Circular LBP with (16, 2) neighborhood
    lbp_16_2 = extended_lbp(image_uint8, radius=2, neighbors=16)

    # Circular LBP with (24, 3) neighborhood
    lbp_24_3 = extended_lbp(image_uint8, radius=3, neighbors=24)

    # Display the original and LBP images in a row
    plt.figure(figsize=(16, 8))
    plt.subplot(1, 4, 1)
    plt.imshow(normalized_image, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1, 4, 2)
    plt.imshow(lbp_8_1, cmap='gray')
    plt.title('LBP (8, 1)')
    plt.axis('off')

    plt.subplot(1, 4, 3)
    plt.imshow(lbp_16_2, cmap='gray')
    plt.title('LBP (16, 2)')
    plt.axis('off')

    plt.subplot(1, 4, 4)
    plt.imshow(lbp_24_3, cmap='gray')
    plt.title('LBP (24, 3)')
```



```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Load the images from the path textures and apply LBP
```

```
texture_directory = 'textures_short'
```

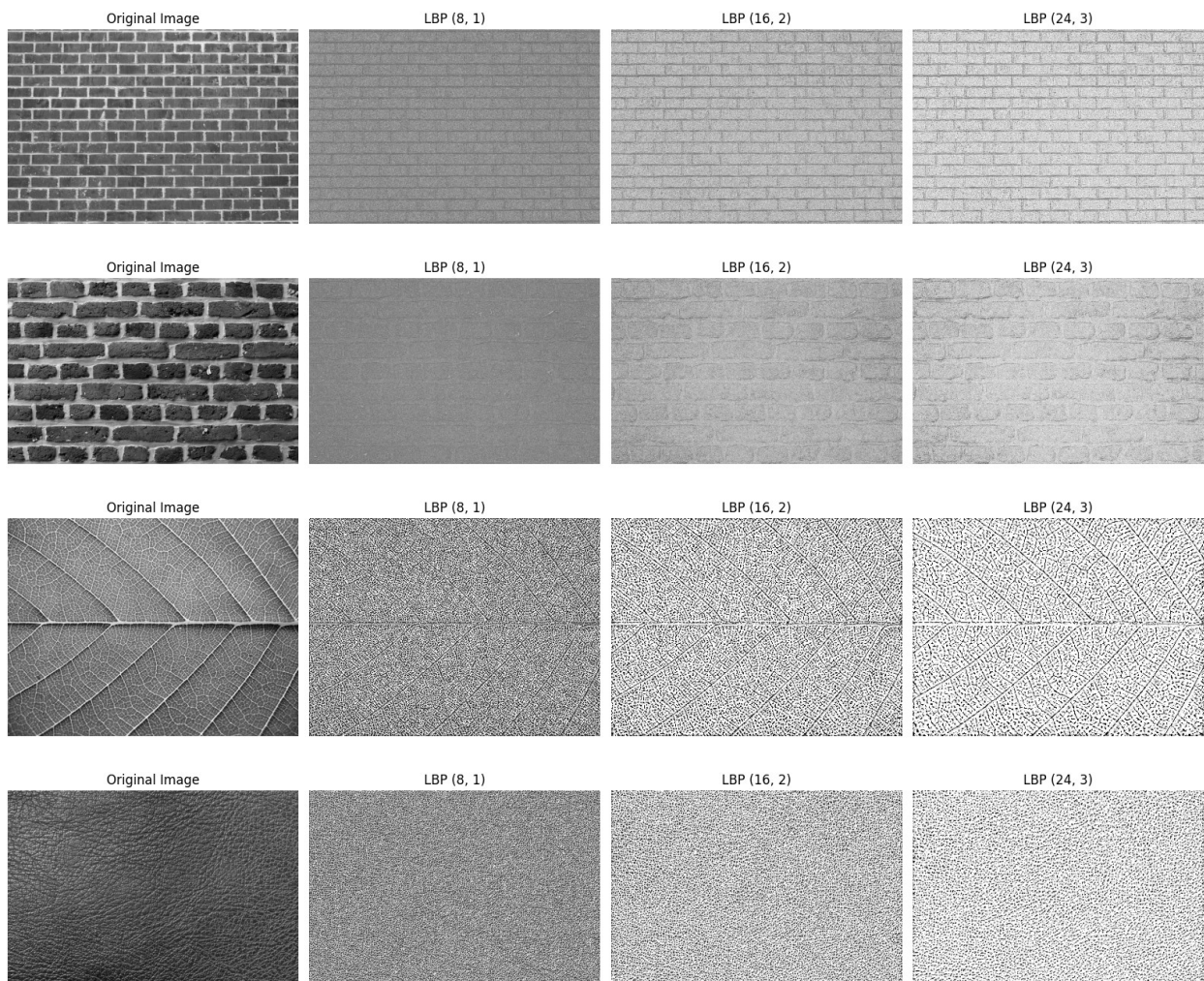
```
for filename in os.listdir(texture_directory):
```

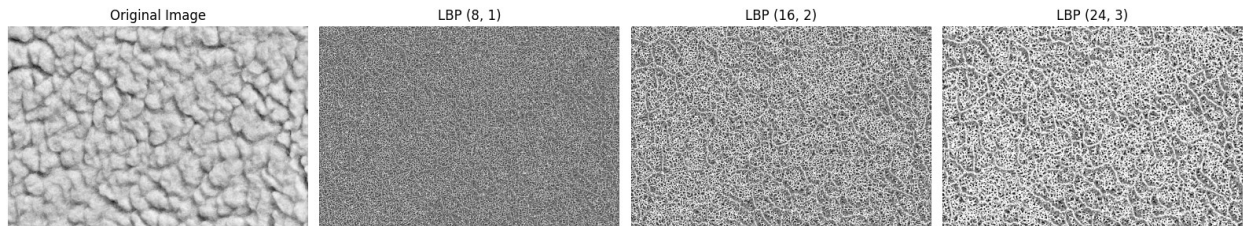
```
    if filename.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp',  
    '.tiff')):
```

```
        image_path = os.path.join(texture_directory, filename)
```

```
        original_image = io.imread(image_path, as_gray=True)
```

```
        apply_lbp_and_display(original_image)
```





A clear pattern can be drawn from all the images on the directory. Since LBP computes local patterns by comparing each pixel with its neighbours, as the parameters are increased (number of neighbours and radius), larger and more structural patterns appear more clear and distinct.

With this visualization, we can say that smaller values of P and R are better for low resolution images where high radius would sample outside the desired areas, and also if finer details are desired to be captured. However, noisy images may be a considerable issue and a previous denoising filter would be needed. On the other hand, when broader features are desired to be captured, larger P and R are generally better.

```
# Function to compute LBP histogram for an image
def compute_lbp_histogram(image):
    # Compute LBP features
    lbp = feature.local_binary_pattern(image, P=8, R=1,
method="uniform")
    # Calculate histogram of LBP features
    hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0, 10),
range=(0, 9))
    # Normalize the histogram to obtain the feature vector
    hist = hist.astype("float")
    hist /= (hist.sum() + 1e-7)
    return hist

# Function to compute squared error between two LBP feature vectors
def compute_squared_error(feature1, feature2):
    return np.sum((feature1 - feature2) ** 2)

def extract_lbp_features_and_compare(directory):
    features = []
    labels = []
    filenames = []

    # Extract features from all images
    for filename in os.listdir(directory):
        if filename.endswith(".jpg") or filename.endswith(".JPG"):
            image_path = os.path.join(directory, filename)

            # Load image using OpenCV
            image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

            hist = compute_lbp_histogram(image)
            # Append the feature vector and label
            features.append(hist)
```

```

        labels.append(filename.split('_')[0]) # Knowing that the
label is the prefix before '_'
        filenames.append(filename)

    features = np.array(features)
    labels = np.array(labels)

    # Compute squared errors between all pairs of images
    print("Pairwise Squared Errors between images:")
    print("-" * 50)

    for i in range(len(features)):
        for j in range(i + 1, len(features)):
            error = compute_squared_error(features[i], features[j])
            print(f"{filenames[i]} vs {filenames[j]}: {error:.6f}")

    return features, labels, filenames

def plot_lbp_comparison_bar(directory):
    features, labels, filenames =
extract_lbp_features_and_compare(directory)

    # Compute squared errors between all pairs of images
    comparisons = []
    errors = []

    for i in range(len(features)):
        for j in range(i + 1, len(features)):
            error = compute_squared_error(features[i], features[j])
            comparison_label = f"{filenames[i]}\nvs\n{filenames[j]}"
            comparisons.append(comparison_label)
            errors.append(error)

    # Sort by error values (lowest to highest)
    sorted_indices = np.argsort(errors)
    sorted_comparisons = [comparisons[i] for i in sorted_indices]
    sorted_errors = [errors[i] for i in sorted_indices]

    # Create bar plot
    plt.figure(figsize=(15, 8))
    bars = plt.bar(range(len(sorted_comparisons)), sorted_errors,
color='skyblue', edgecolor='navy', alpha=0.7)

    # Customize the plot
    plt.xlabel('Image Comparisons', fontsize=12)
    plt.ylabel('Squared Error', fontsize=12)
    plt.title('LBP Feature Comparison - Squared Errors Between Image
Pairs', fontsize=14, fontweight='bold')
    plt.xticks(range(len(sorted_comparisons)), sorted_comparisons,
rotation=45, ha='right', fontsize=10)

```

```

plt.grid(axis='y', alpha=0.3)

# Add value labels on top of bars
for i, (bar, error) in enumerate(zip(bars, sorted_errors)):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
max(sorted_errors)*0.01,
            f'{error:.4f}', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()

return features, labels, filenames, sorted_comparisons,
sorted_errors

def visualize_texture_images(directory):
    # Get all image files
    image_files = []
    for filename in os.listdir(directory):
        if filename.lower().endswith(('.jpg', '.JPG')):
            image_files.append(filename)

    n_images = len(image_files)

    # Create subplot with n columns and 1 row
    fig, axes = plt.subplots(1, n_images, figsize=(4*n_images, 6))

    # Handle single image case
    if n_images == 1:
        axes = [axes]

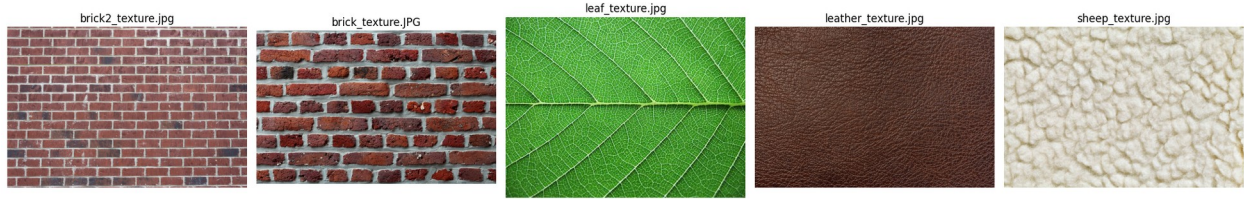
    for i, filename in enumerate(image_files):
        image_path = os.path.join(directory, filename)
        image = io.imread(image_path) # Remove as_gray=True to keep
original colors

        axes[i].imshow(image) # No cmap needed for color images
        axes[i].set_title(filename)
        axes[i].axis('off')

    plt.tight_layout()
    plt.show()

image_directory_short = 'textures_short'
visualize_texture_images(image_directory_short)
features, labels, filenames, comparisons, errors =
plot_lbp_comparison_bar(image_directory_short)
# Print the extracted features and labels
print("LBP Features:")
print(features)

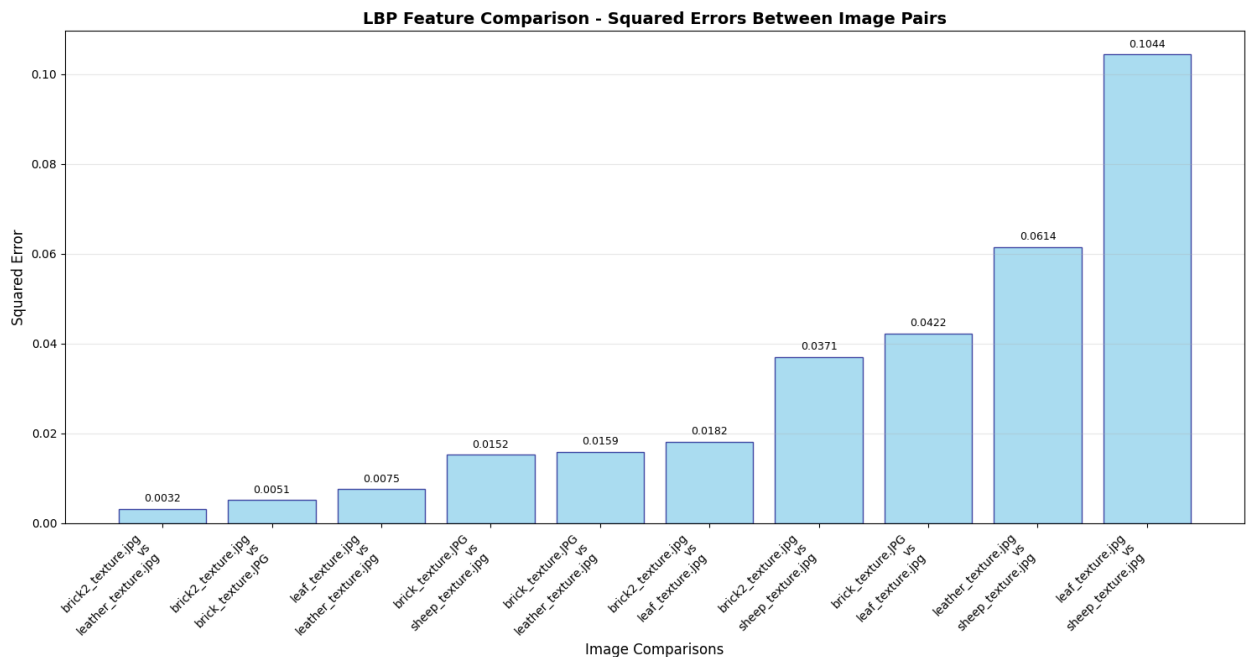
```

Pairwise Squared Errors between images:

```

-----
brick2_texture.jpg vs brick_texture.JPG: 0.005108
brick2_texture.jpg vs leaf_texture.jpg: 0.018170
brick2_texture.jpg vs leather_texture.jpg: 0.003151
brick2_texture.jpg vs sheep_texture.jpg: 0.037053
brick_texture.JPG vs leaf_texture.jpg: 0.042197
brick_texture.JPG vs leather_texture.jpg: 0.015908
brick_texture.JPG vs sheep_texture.jpg: 0.015218
leaf_texture.jpg vs leather_texture.jpg: 0.007529
leaf_texture.jpg vs sheep_texture.jpg: 0.104404
leather_texture.jpg vs sheep_texture.jpg: 0.061442
  
```



LBP Features:

```

[[0.08802002 0.09172282 0.06815755 0.09515747 0.11752238 0.0990153
  0.07148722 0.09365438 0.27526286]
 [0.06933678 0.08646704 0.07414953 0.11347117 0.14887626 0.11705889
  0.08321726 0.08552748 0.2218956 ]
 [0.12915     0.10596667 0.046775    0.04615     0.06585     0.07420833
  0.05746667 0.09968333 0.37475     ]
 [0.09743313 0.09970679 0.07565329 0.07872428 0.09476852 0.0757716
  0.06290123 0.10127058 0.31377058]
  
```



```
[0.03516276 0.06615244 0.06361736 0.14667436 0.21037181 0.1696543
0.0942426 0.06535948 0.1487649 ]]
```

If we analyze the results and see the square error between pairs, we can see that the pair with less square error (more similar for LBP features) is brick2 and leather, which is curious as the images, at first glance are not similar. Additionally, both brick images, which have a similar image composition have a slightly higher error. From this, we can see on the graph increasing errors as the textures are less and less similar being leaf and sheep the least similar in terms of square error. It is also important to note that these are with a fixed P and R values for LBP.

Task 4: Image Alignment Using Feature Matching

```
# Estimate transformation matrix using RANSAC and align images
def estimate_transformation_and_align(image1, image2,
    transformation_type='homography'):

    # Find SIFT matches
    matches, kp1, kp2, desc1, desc2 = match_sift_features(image1,
        image2, ratio_threshold=0.7, visualize=False)

    if len(matches) < 4:
        print(f"Not enough matches found: {len(matches)}. Need at
            least 4 for homography.")
        return None, None, None, None

    # Extract corresponding points
    src_pts = np.float32([kp1[m.queryIdx].pt for m in
        matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in
        matches]).reshape(-1, 1, 2)

    print(f"Using {len(matches)} matches for transformation
        estimation")

    # Estimate transformation matrix using RANSAC
    if transformation_type == 'homography':
        # Homography for perspective changes
        transformation_matrix, mask = cv2.findHomography(
            src_pts, dst_pts,
            cv2.RANSAC,
            ransacReprojThreshold=5.0,
            confidence=0.99,
            maxIter=2000
        )
        transform_name = "Homography"
    else:
        # Affine transformation for simpler cases
        transformation_matrix, mask = cv2.estimateAffinePartial2D(
            src_pts, dst_pts,
```

```

        cv2.RANSAC,
        ransacReprojThreshold=5.0,
        confidence=0.99,
        maxIters=2000
    )
    transform_name = "Affine"

    if transformation_matrix is None:
        print(f"Failed to estimate {transform_name} transformation")
        return None, None, None, None

    # Count inliers
    inliers = mask.ravel().tolist()
    num_inliers = sum(inliers)
    print(f"{transform_name} estimation successful:")
    print(f"    - Inliers: {num_inliers}/{len(matches)}")
    print(f"({100*num_inliers/len(matches):.1f}%)")

    # Warp image1 to align with image2
    h2, w2 = image2.shape
    if transformation_type == 'homography':
        aligned_image = cv2.warpPerspective(image1,
        transformation_matrix, (w2, h2))
    else:
        aligned_image = cv2.warpAffine(image1, transformation_matrix,
        (w2, h2))

    return transformation_matrix, aligned_image, matches, mask

# Visualize the before and after alignment results
def visualize_alignment_results(image1, image2, aligned_image,
matches, mask, transform_matrix, transform_type):

    # Create figure with subplots
    fig = plt.figure(figsize=(20, 12))

    # 1. Original images side by side
    plt.subplot(2, 3, 1)
    plt.imshow(image1, cmap='gray')
    plt.title('Original Image 1 (Source)', fontsize=12)
    plt.axis('off')

    plt.subplot(2, 3, 2)
    plt.imshow(image2, cmap='gray')
    plt.title('Original Image 2 (Target)', fontsize=12)
    plt.axis('off')

    # 2. Aligned result
    plt.subplot(2, 3, 3)
    plt.imshow(aligned_image, cmap='gray')

```

```

plt.title('Aligned Image 1 → Image 2', fontsize=12)
plt.axis('off')

# 3. Overlay comparison (before alignment)
plt.subplot(2, 3, 4)
# Create overlay by blending images
overlay_before = cv2.addWeighted(
    cv2.resize(image1, (image2.shape[1], image2.shape[0])), 0.5,
    image2, 0.5, 0
)
plt.imshow(overlay_before, cmap='gray')
plt.title('Overlay: Before Alignment', fontsize=12)
plt.axis('off')

# 4. Overlay comparison (after alignment)
plt.subplot(2, 3, 5)
overlay_after = cv2.addWeighted(aligned_image, 0.5, image2, 0.5,
0)
plt.imshow(overlay_after, cmap='gray')
plt.title('Overlay: After Alignment', fontsize=12)
plt.axis('off')

# 5. Inlier matches visualization
plt.subplot(2, 3, 6)
# Filter matches to show only inliers
inlier_matches = [matches[i] for i in range(len(matches)) if
mask[i]]

# Re-detect keypoints for visualization
sift = cv2.SIFT_create()
kp1, _ = sift.detectAndCompute(image1, None)
kp2, _ = sift.detectAndCompute(image2, None)

matched_img = cv2.drawMatches(
    image1, kp1, image2, kp2,
    inlier_matches[:50], # Show first 50 inlier matches
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)
plt.imshow(matched_img)
plt.title(f'Inlier Matches ({len(inlier_matches)} total)',
fontsize=12)
plt.axis('off')

plt.tight_layout()
plt.show()

# Print transformation matrix

```

```

print(f"\n{transform_type} Transformation Matrix:")
print(transform_matrix)

# Complete demonstration of image alignment process
def compute_image_alignment(img1_path='nd.jpg', img2_path='nd1.jpg'):

    # Load two example images (replace with your own image paths)
    image1 = cv2.imread(img1_path, cv2.IMREAD_GRAYSCALE)
    image2 = cv2.imread(img2_path, cv2.IMREAD_GRAYSCALE)

    print(f"Image 1 shape: {image1.shape}")
    print(f"Image 2 shape: {image2.shape}")

    # Try both transformation types
    for transform_type in ['homography', 'affine']:
        print(f"\nEstimating {transform_type} Transformation")
        # Estimate transformation and align
        transform_matrix, aligned_img, matches, mask =
estimate_transformation_and_align(
            image1, image2, transform_type
        )

        if transform_matrix is not None:
            # Visualize results
            visualize_alignment_results(
                image1, image2, aligned_img, matches, mask,
                transform_matrix, transform_type.capitalize()
            )
        else:
            print(f"Failed to estimate {transform_type}
transformation")

# Run the demonstration
compute_image_alignment()
compute_image_alignment('dwarf1.jpg', 'dwarf2.jpg')

Image 1 shape: (932, 743)
Image 2 shape: (751, 805)

--- HOMOGRAPHY TRANSFORMATION ---
Found 58 good matches
Using 58 matches for transformation estimation
Homography estimation successful:
- Inliers: 18/58 (31.0%)

```

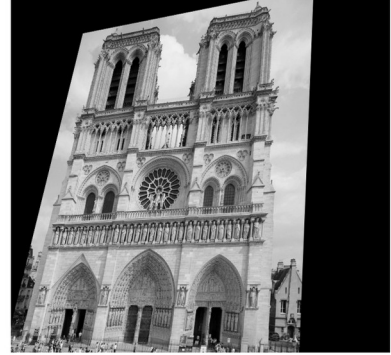
Original Image 1 (Source)



Original Image 2 (Target)



Aligned Image 1 → Image 2



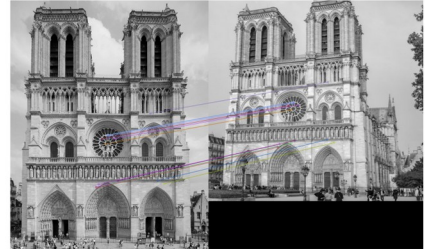
Overlay: Before Alignment



Overlay: After Alignment



Inlier Matches (18 total)



Homography Transformation Matrix:

```
[ [ 5.22833290e-01 -1.76784749e-01 1.52450354e+02]
  [-1.26918625e-01 5.50100114e-01 7.39953002e+01]
  [-2.13204972e-04 -2.41699541e-04 1.00000000e+00] ]
```

--- AFFINE TRANSFORMATION ---

Found 58 good matches

Using 58 matches for transformation estimation

Affine estimation successful:

- Inliers: 12/58 (20.7%)

Original Image 1 (Source)



Original Image 2 (Target)



Aligned Image 1 → Image 2



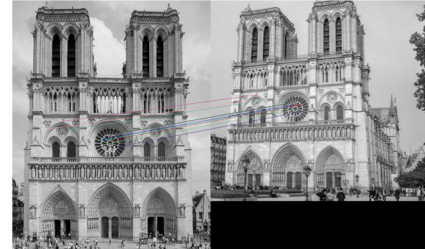
Overlay: Before Alignment



Overlay: After Alignment



Inlier Matches (12 total)



Affine Transformation Matrix:

```
[[ 0.80080254  0.06506526 -9.13336413]
 [-0.06506526  0.80080254  1.42554258]]
```

Image 1 shape: (337, 332)

Image 2 shape: (340, 341)

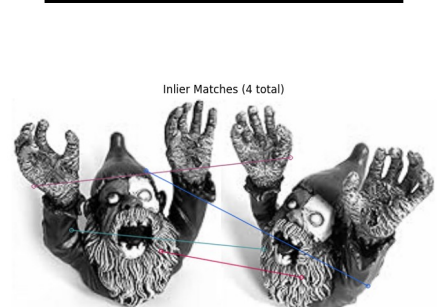
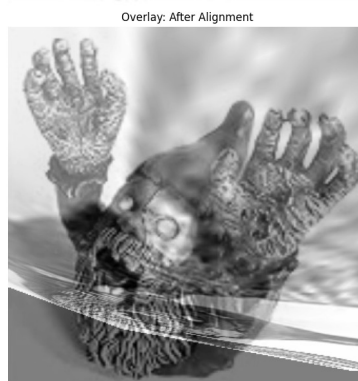
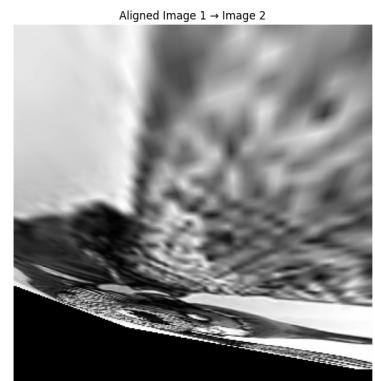
--- HOMOGRAPHY TRANSFORMATION ---

Found 6 good matches

Using 6 matches for transformation estimation

Homography estimation successful:

- Inliers: 4/6 (66.7%)



Homography Transformation Matrix:

```
[[-3.35523824e+00  1.85282612e+00 -1.94924463e+02]
 [-4.95542110e+00 -1.53101692e+00  3.38696901e+02]
 [-1.50567734e-02 -6.99140438e-03  1.00000000e+00]]
```

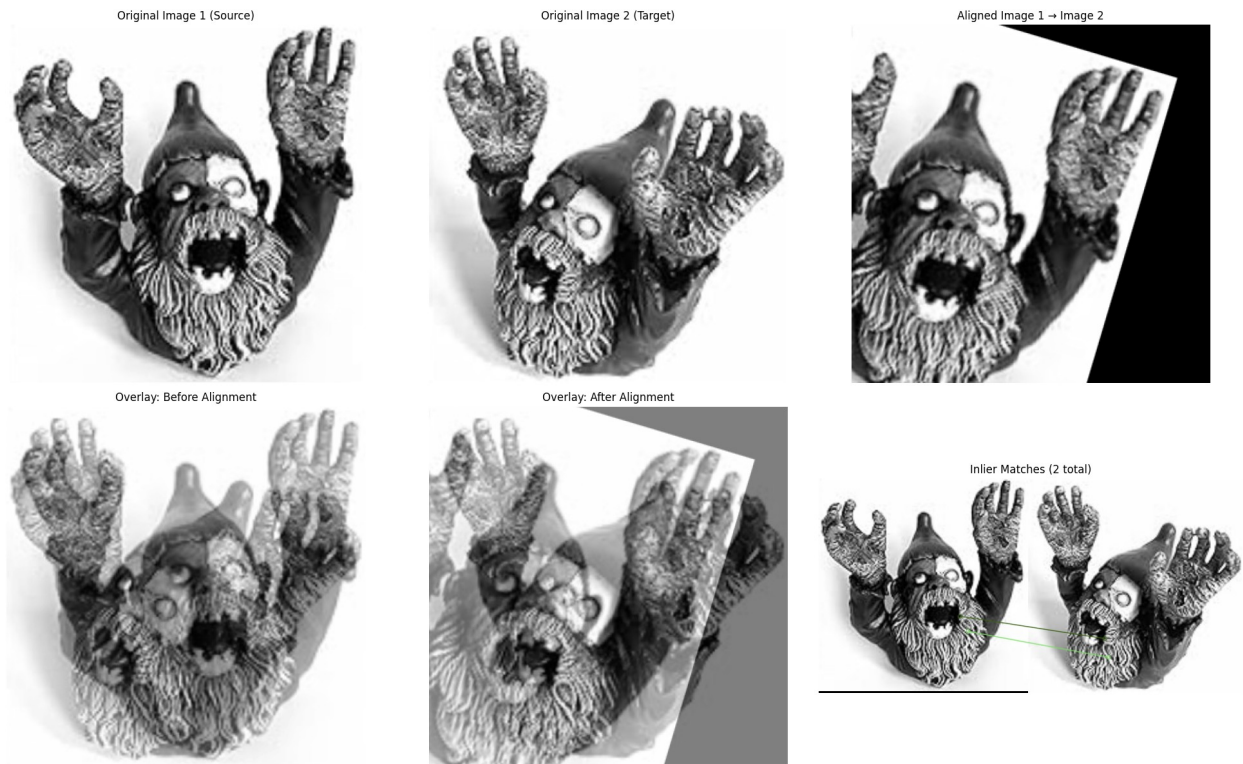
--- AFFINE TRANSFORMATION ---

Found 6 good matches

Using 6 matches for transformation estimation

Affine estimation successful:

- Inliers: 2/6 (33.3%)



Affine Transformation Matrix:

$$\begin{bmatrix} 1.0885596 & -0.32432417 & -51.8426471 \\ 0.32432417 & 1.0885596 & -56.88938402 \end{bmatrix}$$

My approach was to run both affine and homography transformation matrices on the images. Based on the results, it is clear that we cannot draw any general conclusions about whether affine or homography performs better overall, as in some images the affine transformation shows better inlier results, while in others the homography performs better.

Regarding the visual results, in the image of the dwarf, the current approach is able to detect matches but fails to properly align the image. This is likely due to the object being rotated along the Z-axis rather than the X or Y axes, which I believe is the source of the alignment error, as this method may lack the ability to capture 3D correspondences.

On the other hand, in the image of Notre Dame, the results are better. The homography transformation successfully captures the rotation of the cathedral, while the affine approach is more effective at handling perspective changes. In conclusion, a combination of both approaches would likely result in the best results, as it would handle both rotation and perspective correction.

Task 5: Texture Classification Using LBP Features

```
def visualize_texture_images_grid(directory, cols=3):
    # Get all image files
    image_files = []
```

```

for filename in os.listdir(directory):
    if filename.lower().endswith(('.jpg', '.JPG')):
        image_files.append(filename)

if not image_files:
    print("No images found in directory")
    return

n_images = len(image_files)
rows = (n_images + cols - 1) // cols # Calculate rows needed

# Create subplot with calculated rows and specified columns
fig, axes = plt.subplots(rows, cols, figsize=(3*cols, 3*rows))

# Handle single row case
if rows == 1:
    axes = axes.reshape(1, -1)

# Flatten axes for easier indexing
axes_flat = axes.flatten()

for i, filename in enumerate(image_files):
    image_path = os.path.join(directory, filename)
    image = io.imread(image_path)

    axes_flat[i].imshow(image)
    axes_flat[i].set_title(filename, fontsize=8)
    axes_flat[i].axis('off')

# Hide empty subplots
for i in range(n_images, len(axes_flat)):
    axes_flat[i].axis('off')

plt.tight_layout()
plt.show()

def extract_lbp_features_for_classification(directory):
    features = []
    labels = []
    filenames = []

    # Extract features from all images
    for filename in os.listdir(directory):
        if filename.lower().endswith(('.jpg', '.JPG')):
            image_path = os.path.join(directory, filename)

            # Load image using OpenCV
            image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
            if image is None:
                continue

```

```

        # Compute LBP histogram for the image
        hist = compute_lbp_histogram(image)

        # Append the feature vector and label
        features.append(hist)
        # Extract label from filename (assuming format:
class_*.jpg)
        label = filename.split('_')[0] if '_' in filename else
filename.split('.')[0]
        labels.append(label)
        filenames.append(filename)

    return np.array(features), np.array(labels), filenames

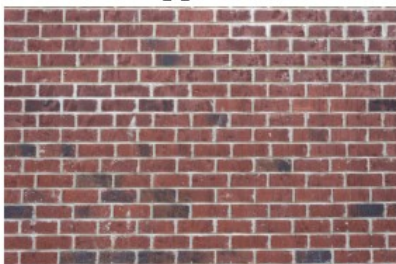
texture_directory = 'textures'
visualize_texture_images_grid(texture_directory, cols=3)

features, labels, filenames =
extract_lbp_features_for_classification(texture_directory)

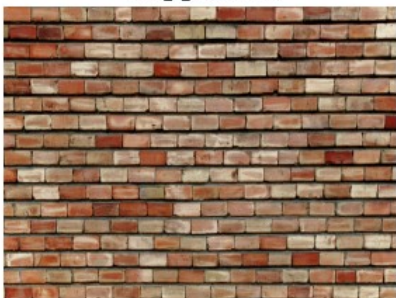
print(f"Extracted features from {len(features)} images")
print(f"Feature vector dimension: {features.shape[1]}")
print(f"Classes found: {np.unique(labels)}")

```

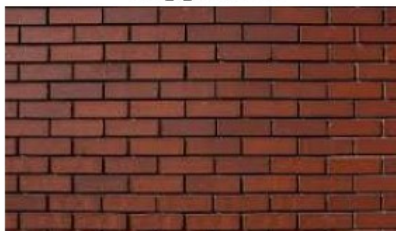

brick_2_texture.jpg



brick_3_texture.jpg



brick_4_texture.jpg



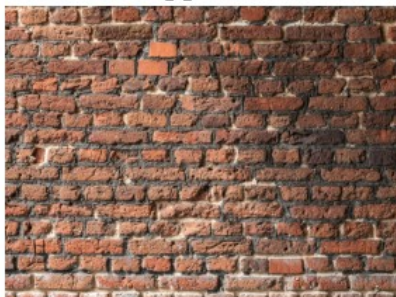
brick_5_texture.jpg



brick_6_texture.jpg



brick_7_texture.jpg



brick_8_texture.jpg



brick_texture.JPG



brick_wall_texture.jpg



brick_zembratiles_texture.jpg



leaf_2_texture.jpg



leaf_3_texture.jpg



leaf_5_texture.jpg



leaf_6_texture.jpg



Extracted features from 16 images
Feature vector dimension: 9
Classes found: ['brick' 'leaf']

In this visualization it is easy to see that the problem is a simple classification problem of two classes, brick and leaf. The dataset is rather small with a total of 16 instances.

```
def train_and_evaluate_knn(features, labels, filenames):  
    # Encode labels to numerical values  
    label_encoder = LabelEncoder()  
    encoded_labels = label_encoder.fit_transform(labels)  
  
    # Check class distribution  
    unique_labels, counts = np.unique(encoded_labels,  
    return_counts=True)  
    print(f"Class distribution: {dict(zip(label_encoder.classes_,  
    counts))}")  
  
    test_size = 0.3  
    X_train, X_test, y_train, y_test, train_files, test_files =  
    train_test_split(  
        features, encoded_labels, filenames,  
        test_size=test_size, random_state=42, stratify=encoded_labels  
    )  
  
    # Train k-NN classifier  
    knn = KNeighborsClassifier(n_neighbors=3) # Ensure k <= training  
    samples  
  
    # Train the classifier  
    knn.fit(X_train, y_train)  
  
    # Make predictions  
    y_pred = knn.predict(X_test)  
  
    # Calculate accuracy  
    accuracy = accuracy_score(y_test, y_pred)  
  
    print(f"\nTraining set size: {len(X_train)}")  
    print(f"Test set size: {len(X_test)}")  
    print(f"Accuracy: {accuracy:.4f}")  
  
    # Print classification report  
    print("\nClassification Report:")  
    print(classification_report(y_test, y_pred,  
    target_names=label_encoder.classes_))  
  
    # Plot confusion matrix  
    cm = confusion_matrix(y_test, y_pred)
```

```

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.title(f'Confusion Matrix - k-NN (k={min(3, len(X_train))})')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

return knn, X_test, y_test, y_pred, test_files, label_encoder,
accuracy

def visualize_classification_examples(knn, X_test, y_test, y_pred,
test_files, label_encoder, directory):
    # Find correctly and incorrectly classified examples
    correct_indices = np.where(y_test == y_pred)[0]
    incorrect_indices = np.where(y_test != y_pred)[0]

    n_correct_show = min(6, len(correct_indices))
    if n_correct_show > 0:
        fig, axes = plt.subplots(2, 3, figsize=(15, 10))
        axes = axes.flatten()

        for i in range(n_correct_show):
            idx = correct_indices[i]
            filename = test_files[idx]
            image_path = os.path.join(directory, filename)
            image = io.imread(image_path)

            true_label =
label_encoder.inverse_transform([y_test[idx]])[0]
            pred_label =
label_encoder.inverse_transform([y_pred[idx]])[0]

            axes[i].imshow(image)
            axes[i].set_title(f'File: {filename}\nTrue: {true_label},
Pred: {pred_label}', fontsize=10)
            axes[i].axis('off')

        # Hide empty subplots
        for i in range(n_correct_show, 6):
            axes[i].axis('off')

        plt.suptitle('Correctly Classified Examples', fontsize=16)
        plt.tight_layout()
        plt.show()

    n_incorrect_show = min(6, len(incorrect_indices))
    if n_incorrect_show > 0:

```

```

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

for i in range(n_incorrect_show):
    idx = incorrect_indices[i]
    filename = test_files[idx]
    image_path = os.path.join(directory, filename)
    image = io.imread(image_path)

    true_label =
label_encoder.inverse_transform([y_test[idx]])[0]
    pred_label =
label_encoder.inverse_transform([y_pred[idx]])[0]

    axes[i].imshow(image)
    axes[i].set_title(f'File: {filename}\nTrue: {true_label},
Pred: {pred_label}', fontsize=10, color='red')
    axes[i].axis('off')

# Hide empty subplots
for i in range(n_incorrect_show, 6):
    axes[i].axis('off')

plt.suptitle('Incorrectly Classified Examples', fontsize=16,
color='red')
plt.tight_layout()
plt.show()

```

Train and evaluate the k-NN classifier

```
knn, X_test, y_test, y_pred, test_files, label_encoder, accuracy =
train_and_evaluate_knn(features, labels, filenames)
```

Visualize classification examples

```
visualize_classification_examples(knn, X_test, y_test, y_pred,
test_files, label_encoder, texture_directory)
```

```
Class distribution: {np.str_('brick'): np.int64(9), np.str_('leaf'):
np.int64(7)}
```

Training set size: 11

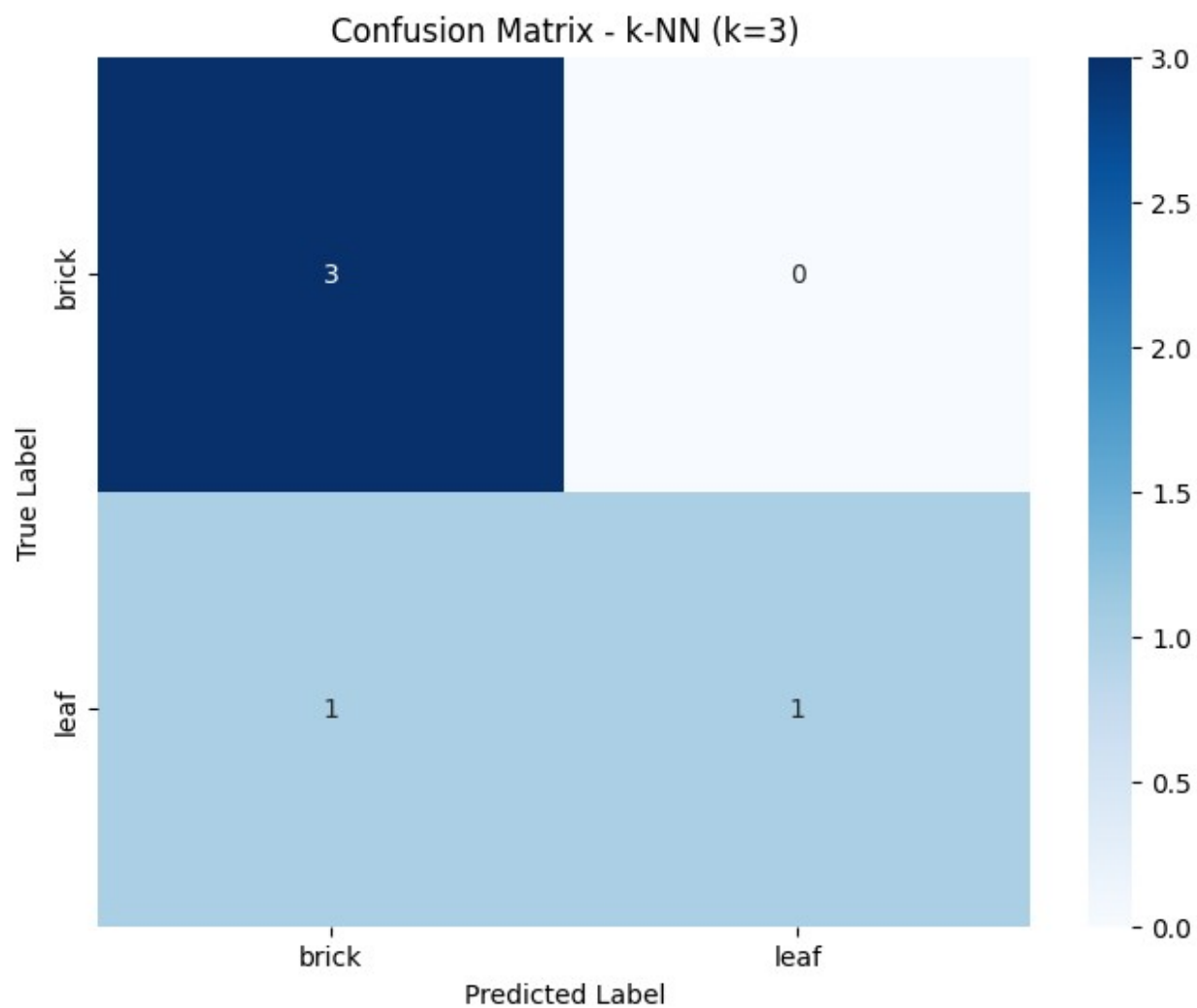
Test set size: 5

Accuracy: 0.8000

Classification Report:

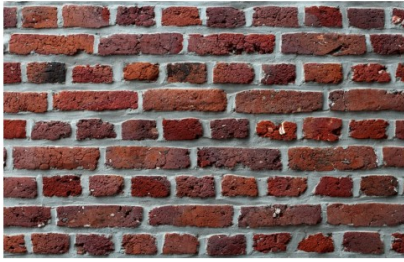
	precision	recall	f1-score	support
brick	0.75	1.00	0.86	3
leaf	1.00	0.50	0.67	2

accuracy			0.80	5
macro avg	0.88	0.75	0.76	5
weighted avg	0.85	0.80	0.78	5



Correctly Classified Examples

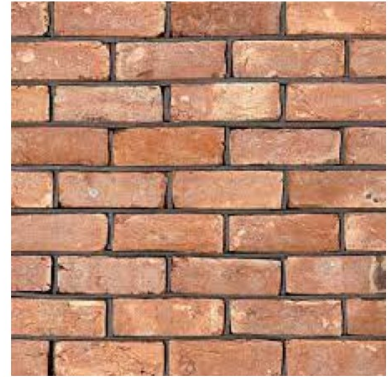
File: brick_texture.JPG
True: brick, Pred: brick



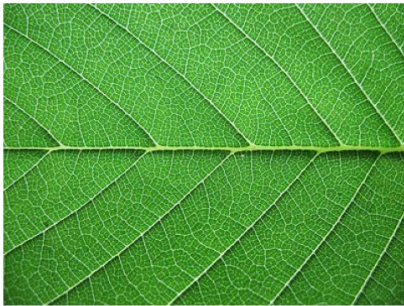
File: brick_5_texture.jpg
True: brick, Pred: brick



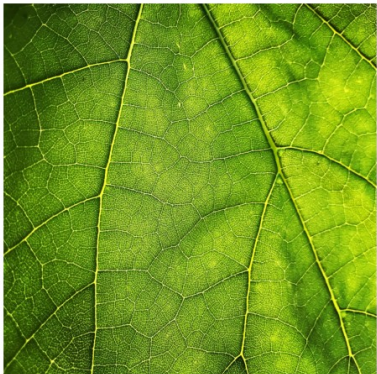
File: brick_6_texture.jpg
True: brick, Pred: brick



File: leaf_texture.jpg
True: leaf, Pred: leaf



File: leaf_5_texture.jpg
True: leaf, Pred: brick



Incorrectly Classified Examples

Looking at the results of the classification problem, we can see that we obtain an accuracy of 0.8 which is quite decent considering the size of the dataset and the simplicity of the approach. However, it is also relevant to mention that initially I had a multi class classification problem where I had 4 classes and the dataset size was proportional to the actual size.

With the multi class problem, the classification accuracy was less than 0.5 which meant that this same approach with more classes and a reduced dataset is not a valid approach.

To solve this, increasing the dataset size would probably increase the performance of this model in the current case of a binary classification problem. Also, the current two classes present in this exercise are from images that are highly different, therefore, it was not complex for the model to differentiate between classes.

Having said this, it is relevant to mention that it is not computationally expensive to train this model and to obtain the features for the images. Therefore, I would say that the results are satisfactory for this task 5.